

K L Deemed to be University

Department of Computer Science and Engineering | 2026–2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

SKILLING SESSION 2 – ENTITY-RELATIONSHIP MODELLING

Name	Chakrika Panakala	Roll No.	2520090042
Course	25CS1302E	CO Mapping	CO1
Session	Skilling 2	Topic	ER Modelling

Aim

To model the Task Hub project with entities, attributes and relationships of every cardinality (1:1, 1:N, M:N), participation constraints, a weak entity, and the extended ER features specialisation/generalisation and aggregation, and then map the ER diagram to relational tables.

ER Modelling Notation

Rectangle = entity; double rectangle = weak entity; diamond = relationship; double diamond = identifying relationship; ellipse = attribute; underlined attribute = key; dashed attribute = partial key; thick line = total participation; ISA (d) = disjoint specialisation; dashed line = relationship on an aggregated relationship.

1. ER-to-Relational Mapping

The Task Hub ER model is converted into relational tables using standard mapping rules. Strong entities become tables, 1:1 relationships use a UNIQUE foreign key, 1:N relationships use a foreign key on the N-side, M:N relationships use separate junction tables, and weak entities use the owner's key together with a partial key.

2. Database and Strong Entity: USER

```
DROP DATABASE IF EXISTS taskhub_db;
CREATE DATABASE taskhub_db;
\c taskhub_db

CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    role VARCHAR(10) NOT NULL CHECK (role IN ('ADMIN', 'MEMBER'))
);
```

3. Specialisation / Generalisation: USER ISA {ADMIN, MEMBER}

The USER hierarchy is disjoint and total. One table is created for each subclass, sharing the superclass primary key.

```
CREATE TABLE admins (
    user_id INT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
    access_level INT NOT NULL DEFAULT 1 CHECK (access_level BETWEEN 1 AND 3)
);

CREATE TABLE members (
    user_id INT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
    skill_level VARCHAR(12) NOT NULL DEFAULT 'BEGINNER'
);
```

4. 1:1 Relationship – HAS

The HAS relationship is represented by a foreign key with UNIQUE on the total side, ensuring that one user cannot have multiple profiles.

```
CREATE TABLE profiles (
    profile_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL UNIQUE REFERENCES users(user_id) ON DELETE CASCADE,
    bio TEXT,
    github_url VARCHAR(200)
);
```

5. 1:N Relationship – OWNS

The foreign key is placed on the N-side. Since PROJECT has total participation, owner_id is NOT NULL.

```
CREATE TABLE projects (
    project_id SERIAL PRIMARY KEY,
```

```
name VARCHAR(100) NOT NULL,  
owner_id INT NOT NULL REFERENCES users(user_id),  
deadline DATE  
);
```

6. M:N Relationship – WORKS_ON

The M:N relationship is mapped to a separate table with a composite primary key. The relationship attribute member_role is stored in the relationship table.

```
CREATE TABLE project_members (  
    user_id INT REFERENCES users(user_id) ON DELETE CASCADE,  
    project_id INT REFERENCES projects(project_id) ON DELETE CASCADE,  
    member_role VARCHAR(30) NOT NULL,  
    joined_on DATE NOT NULL DEFAULT CURRENT_DATE,  
    PRIMARY KEY (user_id, project_id)  
);
```

7. Aggregation – ADMIN EVALUATES WORKS_ON

The aggregation represents ADMIN evaluating the WORKS_ON relationship, identified by the user-project pair. A composite foreign key ensures that an evaluation refers to an existing WORKS_ON pair.

```
CREATE TABLE evaluations (
  evaluation_id SERIAL PRIMARY KEY,
  admin_id INT NOT NULL REFERENCES admins(user_id),
  user_id INT NOT NULL,
  project_id INT NOT NULL,
  rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  FOREIGN KEY (user_id, project_id)
    REFERENCES project_members(user_id, project_id)
);
```

8. Relationships – CONTAINS and ASSIGNED_TO

CONTAINS is a 1:N relationship with total participation of TASK, so project_id is NOT NULL. ASSIGNED_TO is N:1 and has partial participation, so assignee_id may be NULL.

```
CREATE TABLE tasks (
  task_id SERIAL PRIMARY KEY,
  project_id INT NOT NULL REFERENCES projects(project_id) ON DELETE CASCADE,
  assignee_id INT REFERENCES users(user_id) ON DELETE SET NULL,
  title VARCHAR(200) NOT NULL,
  status VARCHAR(12) NOT NULL DEFAULT 'TODO'
    CHECK (status IN ('TODO', 'IN_PROGRESS', 'DONE')),
  due_date DATE
);
```

9. Weak Entity – COMMENT

COMMENT is a weak entity owned by TASK. Its primary key is the owner's key task_id together with the partial key comment_no. ON DELETE CASCADE removes comments when the owner task is deleted.

```
CREATE TABLE comments (
  task_id INT REFERENCES tasks(task_id) ON DELETE CASCADE,
  comment_no INT,
  author_id INT NOT NULL REFERENCES users(user_id),
  body TEXT NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (task_id, comment_no)
);
```

10. M:N Relationship – TAGGED

```
CREATE TABLE tags (
  tag_id SERIAL PRIMARY KEY,
  name VARCHAR(30) NOT NULL UNIQUE
);

CREATE TABLE task_tags (
  task_id INT REFERENCES tasks(task_id) ON DELETE CASCADE,
  tag_id INT REFERENCES tags(tag_id) ON DELETE CASCADE,
  PRIMARY KEY (task_id, tag_id)
);
```

11. Sample Data

```
INSERT INTO users (full_name, email, role) VALUES
('Habeeb', 'habeeb@taskhub.dev', 'ADMIN'),
('Ananya Rao', 'ananya@taskhub.dev', 'MEMBER'),
('Rahul Verma', 'rahul@taskhub.dev', 'MEMBER');

INSERT INTO admins (user_id, access_level) VALUES (1, 3);
INSERT INTO members (user_id, skill_level)
VALUES (2, 'INTERMEDIATE'), (3, 'BEGINNER');

INSERT INTO profiles (user_id, bio)
VALUES (1, 'Backend lead'), (2, 'Frontend developer');

INSERT INTO projects (name, owner_id, deadline)
VALUES ('Task Hub Backend', 1, '2026-11-30');

INSERT INTO project_members VALUES
(2, 1, 'Database Engineer', '2026-09-01'),
(3, 1, 'API Developer', '2026-09-05');

INSERT INTO evaluations (admin_id, user_id, project_id, rating)
VALUES (1, 2, 1, 5);

INSERT INTO tasks (project_id, assignee_id, title, status) VALUES
(1, 2, 'Create PostgreSQL schema', 'IN_PROGRESS'),
(1, 3, 'Write FastAPI CRUD', 'TODO');
```

12. Verification and Output

Foreign-key verification

The database catalog confirmed 14 foreign-key relationships involving admins, comments, evaluations, members, profiles, project_members, projects, task_tags and tasks.

Weak entity verification

The comments table contains the composite primary key (task_id, comment_no), confirming the weak-entity mapping.

1:1 relationship verification

Attempting to insert a second profile for user 1 produced a duplicate-key error on the UNIQUE constraint, confirming the 1:1 relationship.

Aggregation verification

An evaluation for the pair (user_id, project_id) = (1,1) was rejected because that WORKS_ON pair does not exist in project_members.

Weak-entity deletion verification

After deleting task 1, the query returned comments_left_for_task_1 = 0, confirming ON DELETE CASCADE.

M:N relationship query

full_name	project	member_role	rating
-----+-----+-----+-----			
Ananya Rao	Task Hub Backend	Database Engineer	5
Rahul Verma	Task Hub Backend	API Developer	

Result

The ER diagram was mapped to 11 tables using standard rules: strong entities became tables, the 1:1 HAS relationship became a UNIQUE foreign key, 1:N relationships became foreign keys on the N side with NOT NULL for total participation, M:N relationships became junction tables with composite keys, the weak entity COMMENT received the composite key (task_id, comment_no) with ON DELETE CASCADE, the ISA hierarchy became one table per subclass, and the aggregation was enforced by a composite foreign key to project_members. The verification statements confirmed that the mapping rules are enforced by the database.